

Java Backend Development

Spring Boot • Microservices • Kafka • JPA • REST • Security • System Design

150 Scenario-Based Interview Questions + Code Snippets + Senior-Level Answers

Designed for experienced Java backend engineers preparing for senior, lead and technical-specialist interviews. Each answer emphasizes production reasoning, trade-offs and implementation patterns.

How to use this guide

- 1. Core Java & OOP — 15 questions
- 2. Java 8+ Streams, Lambda & Functional Interfaces — 15 questions
- 3. Spring Core & Spring Boot — 15 questions
- 4. REST APIs & Web Backend — 10 questions
- 5. Microservices Architecture — 15 questions
- 6. Kafka & Event-Driven Systems — 10 questions
- 7. Security: JWT, OAuth2 & API Protection — 10 questions
- 8. Database, JPA/Hibernate & SQL — 10 questions
- 9. Resilience, Performance & Production Troubleshooting — 10 questions
- 10. Testing, CI/CD, Docker & Kubernetes — 10 questions
- 11. System Design Scenarios — 10 questions
- 12. Advanced Senior-Level Scenarios — 20 questions
- Final section: 10 mock interview scenarios with ideal answer structures

1. Core Java & OOP

Senior interviews expect more than definitions. Explain the runtime behavior, trade-offs, thread-safety implications and where the concept appears in production code.

1. What is the difference between == and equals()?

Answer / Explanation: == compares primitive values or object references. equals() is intended for logical equality and can be overridden. If equals() is overridden, hashCode() must also be overridden consistently.

```
String a = new String("Java");
String b = new String("Java");
System.out.println(a == b);      // false
System.out.println(a.equals(b)); // true
```

2. Why is String immutable in Java?

Answer / Explanation: Immutability makes String safe to share, supports the String pool, allows cached hash codes, and prevents accidental modification of values used in security-sensitive operations such as class loading and URLs.

```
String s = "Java";
s.concat(" 17");
System.out.println(s); // Java
```

3. Explain the equals/hashCode contract.

Answer / Explanation: If two objects are equal according to equals(), they must return the same hashCode(). Unequal objects may still collide. Violating this rule causes incorrect behavior in HashMap and HashSet.

```
@Override
public boolean equals(Object o) {
    return o instanceof Employee e && id == e.id;
}
@Override
public int hashCode() {
    return Integer.hashCode(id);
}
```

4. How does HashMap work internally?

Answer / Explanation: HashMap computes a hash, maps it to a bucket, and stores entries there. In modern Java, heavily populated buckets can be treeified into balanced trees, improving worst-case lookup behavior. Resizing occurs when the load threshold is crossed.

```
Map<String,Integer> map = new HashMap<>();
map.put("A", 10);
map.get("A");
```

5. HashMap vs ConcurrentHashMap?

Answer / Explanation: HashMap is not thread-safe. ConcurrentHashMap is designed for concurrent access and avoids a single global lock for ordinary reads and updates. It also does not permit null keys or null values.

```
ConcurrentHashMap<String,Integer> counts = new ConcurrentHashMap<>();
counts.merge("java", 1, Integer::sum);
```

6. ArrayList vs LinkedList?

Answer / Explanation: ArrayList provides fast indexed access and usually better cache locality. LinkedList has cheap insertion/removal only when you already have the node position; random access is O(n). In most application code, ArrayList is the better default.

```
List<String> users = new ArrayList<>();
users.add("Pavan");
String first = users.get(0);
```

7. What is immutability and how do you design an immutable class?

Answer / Explanation: Make the class final, fields private and final, initialize through the constructor, expose no setters, and defensively copy mutable objects such as Date or collections.

```
public final class Money {
    private final BigDecimal amount;
    public Money(BigDecimal amount) { this.amount = amount; }
    public BigDecimal getAmount() { return amount; }
}
```

8. Interface vs abstract class?

Answer / Explanation: Use an interface to define a capability or contract that unrelated classes can implement. Use an abstract class when you need shared state or implementation. Modern interfaces can also contain default and static methods.

```
interface PaymentGateway {
    PaymentResult pay(PaymentRequest request);
}
abstract class BaseService {
    protected void audit() {}
}
```

9. Composition vs inheritance?

Answer / Explanation: Prefer composition when behavior can vary independently. Inheritance creates a tighter coupling and should model a genuine is-a relationship. Composition is often easier to test and evolve.

```
class OrderService {
    private final PaymentGateway gateway;
    OrderService(PaymentGateway gateway) {
        this.gateway = gateway;
    }
}
```

10. What is the Liskov Substitution Principle?

Answer / Explanation: Subtypes should be usable wherever the base type is expected without breaking correctness. A classic violation is making a subclass reject operations that the parent contract promises.

```
// Prefer a small capability interface:
interface Readable { String read(); }
interface Writable { void write(String s); }
```

11. What is Optional and where should it be used?

Answer / Explanation: Optional communicates possible absence in return values. It is useful for repository/service results, but generally should not be used for entity fields, method parameters, or serialization models.

```
Optional<User> user = repository.findById(id);
return user.orElseThrow(() ->
    new UserNotFoundException(id));
```

12. Checked vs unchecked exceptions?

Answer / Explanation: Checked exceptions are enforced by the compiler and suit recoverable conditions when callers can reasonably handle them. Runtime exceptions usually indicate programming errors, invalid state, or domain failures that should be handled centrally.

```
try {
    Files.readString(path);
} catch (IOException ex) {
    throw new FileReadException(ex);
}
```

13. What causes ConcurrentModificationException?

Answer / Explanation: Fail-fast iterators can detect structural modification while iterating. Modify through the iterator or use collection APIs designed for the use case.

```
for (Iterator<String> it = list.iterator(); it.hasNext();) {  
    if (it.next().isBlank()) it.remove();  
}
```

14. What is the difference between final, finally and finalize?

Answer / Explanation: final prevents reassignment/overriding/inheritance depending on usage. finally is a block normally used for cleanup. finalize was an old GC-related mechanism and should not be relied upon; use try-with-resources and explicit lifecycle management.

```
final int MAX = 100;  
try { work(); }  
finally { cleanup(); }
```

15. What is a record in modern Java?

Answer / Explanation: A record is a concise carrier for immutable data with generated accessors, equals, hashCode and toString. It is especially useful for API DTOs and projections.

```
public record CustomerDto(Long id, String name) {}
```

2. Java 8+ Streams, Lambda & Functional Interfaces

Expect coding plus reasoning: explain laziness, intermediate vs terminal operations, side effects, parallel streams, collectors and how to avoid processing huge datasets in memory.

16. What is a functional interface?

Answer / Explanation: It has exactly one abstract method and can be implemented with a lambda. Predicate, Function, Consumer and Supplier are standard examples.

```
@FunctionalInterface
interface Calculator {
    int apply(int a, int b);
}
Calculator add = (a,b) -> a + b;
```

17. Predicate vs Function vs Consumer vs Supplier?

Answer / Explanation: Predicate<T> returns boolean; Function<T,R> transforms a value; Consumer<T> accepts a value and returns nothing; Supplier<T> produces a value without an input.

```
Predicate<Integer> even = n -> n % 2 == 0;
Function<String,Integer> len = String::length;
Consumer<String> log = System.out::println;
Supplier<UUID> id = UUID::randomUUID;
```

18. Explain map vs flatMap.

Answer / Explanation: map transforms one element into one result. flatMap transforms each element into a stream and flattens all resulting streams into one stream.

```
List<List<String>> data = List.of(
    List.of("A","B"), List.of("C"));
List<String> flat = data.stream()
    .flatMap(List::stream)
    .toList();
```

19. Find the second-highest distinct salary.

Answer / Explanation: Map to salary, remove duplicates, sort descending, skip the maximum and take the next value. For very large datasets, consider database-side ranking instead.

```
Optional<Integer> second = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();
```

20. Group employees by department.

Answer / Explanation: Collectors.groupingBy creates a map from department to employees. The downstream collector can further count, sum or select values.

```
Map<String,List<Employee>> byDept =
    employees.stream().collect(
        Collectors.groupingBy(Employee::getDepartment));
```

21. Find highest-paid employee in each department.

Answer / Explanation: Use groupingBy with maxBy. In production, if data originates in a database, a SQL window function may be more efficient.

```
Map<String,Optional<Employee>> result =
    employees.stream().collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(Comparator.comparing(Employee::getSalary))
    ));
```

22. How do you remove duplicates while preserving order?

Answer / Explanation: Use `distinct()` on a sequential ordered stream or a `LinkedHashSet` when materializing the result. `distinct` relies on `equals/hashCode`.

```
List<String> unique = names.stream()
    .distinct()
    .toList();
```

23. What is lazy evaluation in streams?

Answer / Explanation: Intermediate operations such as `filter` and `map` do not execute immediately. They execute when a terminal operation such as `collect`, `count` or `findFirst` requests elements.

```
long count = names.stream()
    .filter(n -> n.startsWith("P"))
    .count();
```

24. When should you avoid `parallelStream()`?

Answer / Explanation: Avoid it for small collections, blocking I/O, shared mutable state, order-sensitive logic, or workloads already using managed thread pools. Parallelism should be measured, not assumed.

```
long count = values.parallelStream()
    .filter(this::expensiveCheck)
    .count();
```

25. Explain `reduce()` with an example.

Answer / Explanation: `reduce` combines stream elements into a single result. The accumulator should be associative for parallel execution to remain correct.

```
int total = numbers.stream()
    .reduce(0, Integer::sum);
```

26. How would you process a large database result?

Answer / Explanation: Do not load millions of entities into a list merely to stream them. Prefer pagination, keyset pagination, database aggregation, JDBC streaming, or controlled batch processing depending on the use case.

```
repository.findAllByStatus(status, PageRequest.of(0, 500));
```

27. What is a side effect in a stream and why is it risky?

Answer / Explanation: Mutating external state inside `map/filter/forEach` makes reasoning and parallel execution harder. Prefer pure transformations and collectors.

```
// Prefer:
List<String> result = names.stream()
    .filter(String::isBlank)
    .toList();
```

28. What is `Collectors.toMap()` gotcha?

Answer / Explanation: Duplicate keys cause `IllegalStateException` unless a merge function is supplied.

```
Map<Long, String> names = employees.stream()
    .collect(Collectors.toMap(
        Employee::getId,
        Employee::getName,
        (oldV, newV) -> oldV));
```

29. `CompletableFuture`: run two calls in parallel.

Answer / Explanation: Use independent futures and combine them. In production, use a bounded executor and define timeouts rather than relying on an unbounded/common pool for blocking calls.

```
CompletableFuture<A> a = supplyAsync(this::callA, executor);
```

```
CompletableFuture<B> b = supplyAsync(this::callB, executor);
CompletableFuture<Result> r =
    a.thenCombine(b, Result::new);
```

30. How do you handle CompletableFuture failures?

Answer / Explanation: Use exceptionally for fallback transformation, handle when you need both result and exception, and whenComplete for side-effecting observation.

```
future.exceptionally(ex -> fallback())
    .thenApply(this::mapResult);
```

3. Spring Core & Spring Boot

Be ready to explain bean creation, dependency injection, auto-configuration, scopes, profiles, configuration properties, validation and what actually happens during application startup.

31. What is Dependency Injection?

Answer / Explanation: The container creates dependencies and supplies them to a component instead of the component constructing them itself. Constructor injection is preferred because dependencies are explicit and can be final.

```
@Service
class OrderService {
    private final PaymentClient client;
    OrderService(PaymentClient client) {
        this.client = client;
    }
}
```

32. Why prefer constructor injection over field injection?

Answer / Explanation: It makes required dependencies explicit, supports immutability, simplifies unit testing and prevents partially initialized objects.

```
class UserService {
    private final UserRepository repo;
    UserService(UserRepository repo) { this.repo = repo; }
}
```

33. What does @SpringBootApplication contain?

Answer / Explanation: It combines configuration, component scanning and auto-configuration through @Configuration, @ComponentScan and @EnableAutoConfiguration.

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

34. Explain Spring bean scopes.

Answer / Explanation: Singleton is the default and creates one bean per application context. Prototype creates a new instance when requested. Web scopes such as request and session are tied to web lifecycles.

```
@Bean
@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
PaymentContext paymentContext() {
    return new PaymentContext();
}
```

35. What happens when a Spring Boot application starts?

Answer / Explanation: SpringApplication creates the application context, discovers configuration/components, creates and wires beans, starts infrastructure such as the embedded web server, and publishes lifecycle events.

```
SpringApplication.run(Application.class, args);
```

36. @ConfigurationProperties vs @Value?

Answer / Explanation: Use @ConfigurationProperties for grouped, type-safe configuration with validation. @Value is convenient for a small number of properties.

```
@ConfigurationProperties("payment")
public record PaymentProperties(Duration timeout,
                               URI baseUrl) {}
```

37. How do profiles work?

Answer / Explanation: Profiles allow environment-specific beans/configuration. Typical profiles are dev, test, staging and prod. Keep secrets outside source control.

```
@Profile("prod")
@Bean
DataSource productionDataSource() {
    return buildDataSource();
}
```

38. How do you validate REST request data?

Answer / Explanation: Use Bean Validation annotations on DTOs and @Valid/@Validated at the API boundary. Return a consistent error model from global exception handling.

```
public record CreateUserRequest(
    @NotBlank String name,
    @Email String email) {}

@PostMapping
ResponseEntity<> create(@Valid @RequestBody CreateUserRequest req) {
    return ResponseEntity.ok(service.create(req));
}
```

39. How do you implement global exception handling?

Answer / Explanation: Use @RestControllerAdvice with focused @ExceptionHandler methods. Avoid leaking stack traces or internal database details to clients.

```
@RestControllerAdvice
class ApiExceptionHandler {
    @ExceptionHandler(NotFoundException.class)
    ResponseEntity<> notFound(NotFoundException ex) {
        return ResponseEntity.status(404)
            .body(Map.of("code", "NOT_FOUND", "message", ex.getMessage()));
    }
}
```

40. What is Spring Boot Actuator?

Answer / Explanation: Actuator exposes operational endpoints such as health, metrics and info. In production, secure management endpoints and expose only what is required.

```
management.endpoints.web.exposure.include=health,metrics
management.endpoint.health.show-details=when_authorized
```

41. How do you externalize secrets?

Answer / Explanation: Do not hard-code passwords or tokens in application.properties or Git. Use environment variables, Kubernetes Secrets, a cloud secret manager or Vault, with appropriate rotation and access controls.

```
String token = System.getenv("PAYMENT_API_TOKEN");
```

42. How do @Transactional boundaries work?

Answer / Explanation: Spring typically applies transactions through proxies. Put the transaction boundary around the service operation that represents the business unit of work. Be careful with self-invocation and checked-exception rollback rules.

```
@Transactional
public void transfer(Long from, Long to, BigDecimal amount) {
    debit(from, amount);
    credit(to, amount);
}
```

43. What is transaction propagation?

Answer / Explanation: Propagation controls how a method participates in an existing transaction. REQUIRED joins or creates one; REQUIRES_NEW suspends the current transaction and starts a new one.

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void audit(String message) {
    auditRepository.save(new Audit(message));
}
```

44. What is transaction isolation?

Answer / Explanation: Isolation controls visibility of concurrent transactions. Common levels are READ_COMMITTED, REPEATABLE_READ and SERIALIZABLE. Choose based on consistency requirements and database behavior.

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public void process() { /* ... */ }
```

45. What is the N+1 query problem in JPA?

Answer / Explanation: Loading a list of parent entities and lazily accessing a child for each parent can generate one query for the parent list plus N child queries. Solve with fetch joins, EntityGraph, projections or batch fetching.

```
@Query("select o from Order o join fetch o.customer")
List<Order> findOrdersWithCustomer();
```

4. REST APIs & Web Backend

Interviewers increasingly ask about API correctness: idempotency, pagination, versioning, HTTP semantics, security, error contracts and observability.

46. POST vs PUT vs PATCH?

Answer / Explanation: POST commonly creates a subordinate resource or triggers a non-idempotent operation. PUT replaces a resource at a known URI and is intended to be idempotent. PATCH partially updates a resource.

```
POST    /orders
PUT     /customers/42
PATCH  /customers/42
```

47. What does idempotency mean?

Answer / Explanation: Repeating the same request produces the same intended business outcome. Payment APIs commonly use an Idempotency-Key so client retries do not create duplicate charges.

```
POST /payments
Idempotency-Key: 7c2b-unique-request-id
```

48. Design an idempotent payment endpoint.

Answer / Explanation: Persist the idempotency key with a unique constraint, status and response. On a repeat request, return the previously recorded outcome. Coordinate state changes transactionally.

```
CREATE UNIQUE INDEX ux_payment_key
ON payment_request(idempotency_key);
```

49. Which HTTP status codes should a REST API commonly use?

Answer / Explanation: 200/201 for successful operations, 204 for successful no-content operations, 400 for invalid requests, 401 for missing/invalid authentication, 403 for forbidden access, 404 for missing resources, 409 for conflicts, 422 where validation semantics require it, and 5xx for server failures.

```
return ResponseEntity.status(HttpStatus.CREATED)
    .body(createdOrder);
```

50. Offset vs cursor pagination?

Answer / Explanation: Offset is simple but becomes slower or inconsistent for large changing datasets. Cursor/keyset pagination uses a stable ordering key and scales better for large feeds.

```
SELECT *
FROM orders
WHERE id > :lastId
ORDER BY id
LIMIT :pageSize;
```

51. How do you version REST APIs?

Answer / Explanation: Common strategies are URI, header or media-type versioning. Choose one consistently and define compatibility/deprecation policies.

```
GET /api/v1/customers/42
GET /api/v2/customers/42
```

52. How do you prevent mass assignment vulnerabilities?

Answer / Explanation: Do not bind arbitrary JSON directly to persistence entities. Use explicit request DTOs and map only allowed fields.

```
public record UpdateUserRequest(
    @NotBlank String displayName) {}
```

53. How do you implement request correlation IDs?

Answer / Explanation: Generate or propagate a unique correlation ID at the edge, put it into MDC/log context, and forward it to downstream calls. This makes distributed troubleshooting much easier.

```
MDC.put("correlationId", requestId);
try { chain.doFilter(req, res); }
finally { MDC.remove("correlationId"); }
```

54. What is CORS?

Answer / Explanation: Cross-Origin Resource Sharing is a browser security mechanism controlling whether a web page from one origin can call another origin. It is not an authentication mechanism.

```
@CrossOrigin(origins = "https://app.example.com")
@RestController
class CustomerController {}
```

55. How do you handle API backward compatibility?

Answer / Explanation: Prefer additive changes, keep old fields for a deprecation window, avoid changing meanings/types, version breaking changes and communicate migration timelines.

```
// Additive:
record CustomerResponse(
    long id, String name, String email) {}
```


62. What is a circuit breaker?

Answer / Explanation: It stops repeated calls to an unhealthy dependency after failures cross a threshold. It transitions between closed, open and half-open states and protects the caller from cascading failure.

```
@CircuitBreaker(name="payment", fallbackMethod="fallback")
PaymentResponse pay(PaymentRequest req) {
    return client.pay(req);
}
```

63. Retry vs circuit breaker?

Answer / Explanation: Retry addresses transient failures; circuit breaker limits calls when the dependency is persistently unhealthy. Combining retries with timeouts and circuit breakers requires careful limits to avoid traffic amplification.

```
retry: 3 attempts
timeout: 2s
circuit-breaker: open after failure threshold
```

64. What is the bulkhead pattern?

Answer / Explanation: Bulkheads isolate resources so one dependency cannot consume all threads/connections. Use separate bounded pools or concurrency limits for critical downstream operations.

```
ExecutorService paymentPool =
    Executors.newFixedThreadPool(20);
```

65. How do you handle distributed transactions?

Answer / Explanation: Avoid distributed 2PC where possible. Use local transactions plus patterns such as Saga, transactional outbox, compensating actions and idempotent consumers.

```
Order -> PENDING
Payment -> SUCCESS
Inventory -> RESERVED
Order -> CONFIRMED
```

66. Explain Saga pattern.

Answer / Explanation: A Saga breaks a business transaction into local transactions. If a later step fails, compensating actions undo the business effect of earlier steps. It can be orchestrated or choreography-based.

```
Create Order
-> Reserve Inventory
-> Charge Payment
-> Confirm Order
```

```
Failure:
-> Refund Payment
-> Release Inventory
```

67. What is eventual consistency?

Answer / Explanation: Different services may temporarily have different views of the business state. The design must define acceptable delays, reconciliation and user-visible states such as PENDING.

```
Order: CONFIRMED
Payment: PROCESSING
UI: "Payment processing"
```

68. What is the transactional outbox pattern?

Answer / Explanation: Write the business change and an event record in the same local database transaction. A publisher later sends the outbox event to the broker, reducing the risk of committing business data while losing the event.

```
BEGIN;
INSERT INTO orders ...;
INSERT INTO outbox(event_type,payload) ...;
COMMIT;
```

69. How do you make consumers idempotent?

Answer / Explanation: Persist a unique event ID or business key before applying side effects. If the same event is received again, detect it and return without repeating the side effect.

```
CREATE UNIQUE INDEX ux_event  
ON processed_event(event_id);
```

70. What is a dead-letter queue/topic?

Answer / Explanation: Messages that cannot be processed after controlled retries can be routed to a DLQ for investigation or replay. Preserve metadata and avoid infinite retry loops.

```
main-topic -> retry-topic -> DLQ
```

6. Kafka & Event-Driven Systems

Know Kafka beyond the producer API: partitions, ordering, consumer groups, offsets, delivery semantics, rebalancing, retries, schemas and operational trade-offs.

71. What is a Kafka topic and partition?

Answer / Explanation: A topic is a logical stream; partitions provide parallelism and ordering within each partition. A record's key can determine its partition.

```
kafkaTemplate.send("orders", orderId, orderCreatedEvent);
```

72. What is a consumer group?

Answer / Explanation: Consumers in the same group share partitions so each partition is processed by one consumer at a time within that group. Different groups receive their own logical copy of the stream.

```
group.id = "payment-service"  
group.id = "analytics-service"
```

73. How is ordering guaranteed in Kafka?

Answer / Explanation: Kafka preserves ordering within a partition, not globally across a topic. Use a stable key such as orderId so related events go to the same partition.

```
send(topic="orders", key=orderId, value=event);
```

74. What is an offset?

Answer / Explanation: An offset identifies a record's position within a partition. Consumers commit offsets so processing can resume after restart.

```
@KafkaListener(topics="orders", groupId="payment")  
void consume(OrderCreated event) { process(event); }
```

75. At-most-once vs at-least-once vs exactly-once?

Answer / Explanation: At-most-once may lose messages but avoids duplicates. At-least-once can redeliver and therefore requires idempotent processing. Exactly-once semantics are more constrained and depend on the full pipeline; do not treat them as a universal guarantee for external side effects.

```
process(event);  
commitOffset(); // commit strategy matters
```

76. How do you handle poison messages?

Answer / Explanation: Use bounded retries with backoff, route permanently failing messages to a DLQ, capture the exception and message metadata, and provide an operational replay process.

```
retry 1 -> retry 2 -> retry 3 -> DLQ
```

77. How do you prevent duplicate Kafka processing?

Answer / Explanation: Design the consumer to be idempotent, use a unique business/event key, and choose appropriate producer/consumer transaction settings. Exactly-once processing does not remove the need to consider external side effects.

```
if (processedEventRepository.exists(event.id())) return;  
process(event);  
processedEventRepository.save(event.id());
```

78. What causes consumer lag?

Answer / Explanation: Lag increases when producers publish faster than consumers can process, partitions are insufficient, downstream calls are slow, rebalances occur, or consumers are unhealthy. Monitor lag and processing latency.

`records-in-rate > records-out-rate => lag grows`

79. Kafka vs RabbitMQ?

Answer / Explanation: Kafka is strong for durable event streams, high throughput, replay and partitioned processing. RabbitMQ is often attractive for flexible routing and traditional work queues. Choose based on delivery, replay, ordering and operational requirements.

Kafka: event stream + replay

RabbitMQ: queue + routing

80. How do you evolve event schemas?

Answer / Explanation: Use schema compatibility rules, optional/additive fields, explicit event versions where needed, and a schema registry when appropriate. Never silently change field meaning.

```
{
  "eventType": "OrderCreated",
  "schemaVersion": 2,
  "orderId": "123"
}
```

7. Security: JWT, OAuth2 & API Protection

Senior backend engineers should explain authentication versus authorization, token lifecycle, service-to-service security, secret management and common API threats.

81. Authentication vs authorization?

Answer / Explanation: Authentication answers who the caller is. Authorization answers what that identity is allowed to do. They should be modeled separately.

```
Authenticated user: pavan@example.com
Role: PAYMENT_ADMIN
Permission: payment:refund
```

82. Explain JWT authentication flow.

Answer / Explanation: The client authenticates, receives a signed access token, and sends it with API requests. The resource server validates signature, expiration, issuer/audience and required authorities.

```
Authorization: Bearer <access-token>
```

83. Access token vs refresh token?

Answer / Explanation: Access tokens are short-lived credentials for API access. Refresh tokens are longer-lived credentials used to obtain new access tokens and should receive stronger protection and rotation controls.

```
access_token: 10-15 min
refresh_token: longer lifetime
```

84. Where should JWT validation happen?

Answer / Explanation: Validation can happen at the gateway and/or each resource service. Each service should not blindly trust an upstream header; defense-in-depth and zero-trust designs validate the token/identity at appropriate trust boundaries.

```
Gateway -> validate
Service -> authorize endpoint action
```

85. How do you secure service-to-service calls?

Answer / Explanation: Use TLS, strong service identity, short-lived credentials, authorization policies and secret rotation. mTLS or workload identity can be used where the platform supports it.

```
HTTPS + service identity + authorization
```

86. What is OAuth2?

Answer / Explanation: OAuth2 is an authorization framework for delegated access. It defines roles and flows for obtaining access tokens; OpenID Connect adds an identity layer on top for authentication.

```
Authorization Code + PKCE
Client -> Authorization Server -> Access Token
```

87. What is CSRF and when does it matter?

Answer / Explanation: CSRF tricks a browser into sending an authenticated request to a target site. It is especially relevant to cookie-based authentication. Pure bearer-token APIs that do not use ambient browser credentials have a different threat model.

```
SameSite cookies + CSRF token
are common defenses for cookie sessions.
```

88. How do you prevent SQL injection?

Answer / Explanation: Use parameterized queries or ORM bind parameters. Never concatenate untrusted input into SQL.

```
@Query("select u from User u where u.email = :email")
User findByEmail(@Param("email") String email);
```

89. What is rate limiting?

Answer / Explanation: Rate limiting restricts request volume per user, token, IP or other identity to protect resources and enforce fair use. Token bucket and leaky bucket are common algorithms.

```
Bucket capacity = 100
Refill = 10 requests/sec
```

90. How should passwords be stored?

Answer / Explanation: Never store plaintext or reversible encryption. Use a slow password hashing algorithm such as Argon2id, bcrypt or scrypt with a suitable cost, unique salt and secure verification.

```
passwordEncoder.matches(rawPassword, storedHash);
```

8. Database, JPA/Hibernate & SQL

Be ready to diagnose performance, locking and transaction problems—not just write repository methods.

91. Lazy vs eager fetching?

Answer / Explanation: Lazy fetching defers loading an association until accessed; eager fetching loads it immediately. Neither is universally best. Control fetch plans explicitly to avoid N+1 queries and oversized result sets.

```
@ManyToOne(fetch = FetchType.LAZY)
private Customer customer;
```

92. What is optimistic locking?

Answer / Explanation: Optimistic locking detects concurrent updates using a version field. If another transaction changes the row first, the update fails instead of silently overwriting it.

```
@Version
private long version;
```

93. What is pessimistic locking?

Answer / Explanation: Pessimistic locking obtains a database lock during the transaction, useful when conflicts are expected and correctness requires serialization. It can reduce concurrency and cause deadlocks if misused.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Account> findById(Long id);
```

94. What is connection pooling?

Answer / Explanation: Opening a database connection is expensive. A pool keeps a bounded number of reusable connections. Configure pool size based on DB capacity and workload, not simply CPU count.

```
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.connection-timeout=2000
```

95. How do indexes improve performance?

Answer / Explanation: Indexes reduce the amount of data the database must scan for suitable predicates/orderings, but add storage and write overhead. Index based on actual query patterns and execution plans.

```
CREATE INDEX idx_orders_customer
ON orders(customer_id);
```

96. What is a composite index?

Answer / Explanation: An index on multiple columns can support queries using the leading columns efficiently. Column order matters.

```
CREATE INDEX idx_order_status_date
ON orders(status, created_at);
```

97. How do you troubleshoot a slow SQL query?

Answer / Explanation: Inspect the execution plan, row estimates, indexes, joins, predicates, sorting, lock waits and data volume. Avoid guessing; measure with EXPLAIN/EXPLAIN ANALYZE where supported.

```
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE customer_id = 42;
```

98. What is database normalization?

Answer / Explanation: Normalization reduces redundancy and update anomalies by separating data into related tables. Production systems sometimes denormalize deliberately for performance or read models.

```
Customer(id, name)
Order(id, customer_id, total)
```

99. How do you handle large updates safely?

Answer / Explanation: Use bounded batches, suitable indexes, transaction sizing and operational windows. Avoid a single enormous transaction that creates excessive locks, logs or undo/redo pressure.

```
UPDATE orders
SET status = 'ARCHIVED'
WHERE created_at < :cutoff
AND status = 'COMPLETED';
```

100. JPA Entity vs DTO?

Answer / Explanation: Entities represent persistence state and relationships. DTOs represent API or use-case contracts. Returning entities directly can expose fields, trigger lazy loading and couple APIs to the database model.

```
public record OrderDto(Long id, String status) {}
```

9. Resilience, Performance & Production Troubleshooting

These scenario questions distinguish senior engineers: establish hypotheses, measure, isolate and mitigate before making risky changes.

101. API latency jumped from 200 ms to 8 seconds. What do you check?

Answer / Explanation: Start with metrics and traces. Compare application, DB and downstream latency; inspect thread/connection pools, CPU, memory, GC, locks and recent deployments. Use correlation IDs to follow one request across services.

```
Client -> Gateway -> Service -> DB
                        -> Downstream API
```

102. CPU is 100% in production. How do you investigate?

Answer / Explanation: Check which process/thread consumes CPU, capture thread dumps, inspect hot methods/profiling data, look for infinite loops, excessive serialization, regex, stream misuse or high request volume. Correlate with deployment and traffic changes.

```
jstack <pid>
# then inspect RUNNABLE threads
```

103. How do you investigate OutOfMemoryError?

Answer / Explanation: Confirm heap usage and GC behavior, capture a heap dump when safe, inspect dominant objects/retained sizes, review caches and unbounded collections, and check whether traffic or a deployment changed allocation patterns.

```
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/tmp/heap.hprof
```

104. What is a thread dump?

Answer / Explanation: It is a snapshot of JVM threads and their states/stacks. It helps identify deadlocks, blocked threads, thread starvation and hot call paths.

```
jstack <pid> > threads.txt
```

105. How do you detect a deadlock?

Answer / Explanation: A thread dump may explicitly report a Java-level deadlock. Look for cycles where thread A waits for a lock held by B while B waits for A. Fix lock ordering or redesign shared locking.

```
synchronized(lockA) {
    synchronized(lockB) { /* ... */ }
}
```

106. What is backpressure?

Answer / Explanation: Backpressure prevents producers from overwhelming consumers. Apply bounded queues, rate limits, reactive demand, batch sizing or broker-based buffering rather than allowing unbounded memory growth.

```
BlockingQueue<Job> queue =
    new ArrayBlockingQueue<>(1000);
```

107. Why can increasing thread-pool size make performance worse?

Answer / Explanation: Too many threads increase context switching, memory use and contention. If the bottleneck is DB connections or downstream I/O, extra threads simply increase pressure.

```
ExecutorService pool =
    Executors.newFixedThreadPool(20);
```

108. What is timeout budgeting?

Answer / Explanation: Each downstream call should have a bounded timeout that fits within the overall request deadline. Retries must fit within that budget rather than extending latency indefinitely.

```
request budget = 2.0s
downstream timeout = 600ms
retries = bounded
```

109. What metrics should a backend expose?

Answer / Explanation: At minimum: request rate, error rate, latency percentiles, saturation, JVM memory/GC, thread pools, DB pool usage, downstream latency, Kafka lag and business KPIs.

```
http_server_requests_seconds
jvm_memory_used_bytes
hikaricp_connections_active
```

110. What is distributed tracing?

Answer / Explanation: Tracing links spans across services using a trace ID. It reveals where time is spent and where failures propagate. OpenTelemetry is a common instrumentation standard.

```
Trace: A1
  Gateway span
    -> Order span
      -> Payment span
```

10. Testing, CI/CD, Docker & Kubernetes

Modern Java backend interviews often include deployment and production-readiness. Know the difference between unit, integration, contract and end-to-end tests, and understand basic container orchestration.

111. Unit vs integration vs end-to-end testing?

Answer / Explanation: Unit tests isolate a small unit and are fast. Integration tests verify collaboration with real infrastructure/components. End-to-end tests validate a user/business flow and are slower and fewer in number.

```
@Test
void shouldCalculateTotal() {
    assertEquals(120, service.total(items));
}
```

112. Mockito @Mock vs @InjectMocks?

Answer / Explanation: @Mock creates a Mockito mock. @InjectMocks creates the class under test and injects available mocks into its constructor/setters/fields. Prefer constructor injection in production code.

```
@Mock PaymentRepository repo;
@InjectMocks PaymentService service;
```

113. What is Testcontainers?

Answer / Explanation: It runs real dependencies such as PostgreSQL or Kafka in disposable containers during tests, reducing differences between mocked and real infrastructure behavior.

```
@Container
static PostgreSQLContainer<?> postgres =
    new PostgreSQLContainer<>("postgres:16");
```

114. What is contract testing?

Answer / Explanation: It verifies that a provider and consumer agree on an API/event contract without requiring every test to run a full end-to-end environment. It is useful for independently deployed services.

```
Consumer expects:
GET /customers/{id}
200 -> { id, name }
```

115. What should a CI pipeline do?

Answer / Explanation: Compile, run unit tests, static analysis, security checks, package the application, build an immutable artifact/container, and optionally deploy to controlled environments with approval gates.

```
checkout
-> build
-> test
-> scan
-> package
-> docker build
-> deploy
```

116. What makes a Docker image production-ready?

Answer / Explanation: Use a small trusted base image, non-root user, pinned/controlled dependencies, predictable entrypoint, health checks where useful, and no secrets baked into the image.

```
FROM eclipse-temurin:21-jre
COPY app.jar /app/app.jar
USER 10001
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

117. What is a Kubernetes Deployment?

Answer / Explanation: It describes desired replicas and manages rollout/rollback of pods for an application. A Service provides stable network access to matching pods.

```
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
```

118. Liveness vs readiness probe?

Answer / Explanation: Liveness indicates whether the container should be restarted. Readiness indicates whether the instance should receive traffic. A service may be alive but not ready because a dependency is initializing.

```
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
```

119. What is rolling deployment?

Answer / Explanation: New instances are started while old ones are gradually removed, keeping capacity available during deployment. Proper readiness checks and backward-compatible changes are important.

```
old pods: 3 -> new pods: 1
then 2 -> 2
then 3 -> 0
```

120. How do you deploy database changes safely?

Answer / Explanation: Use backward-compatible migrations, typically expand-and-contract: add new structures first, deploy code that supports both versions, migrate data, then remove old structures later.

1. ADD nullable column
2. deploy compatible code
3. backfill
4. enforce constraint
5. remove old column

11. System Design Scenarios

Use a repeatable interview structure: requirements -> scale -> APIs -> data model -> components -> consistency -> failure handling -> security -> observability -> deployment. State trade-offs explicitly.

121. Design an Order Management system.

Answer / Explanation: Separate order ownership from payment/inventory. Use synchronous validation where immediate feedback is required and events for downstream processing. Persist order state and use an outbox for reliable event publication.

```
Client
-> Gateway
-> Order Service -> Order DB
                  -> Outbox -> Kafka
                        -> Payment
                        -> Inventory
                        -> Notification
```

122. Design a Payment Service.

Answer / Explanation: Treat payment as a state machine, not a single boolean. Use idempotency keys, immutable transaction/audit records, provider timeouts, reconciliation, secure secret storage and strong observability.

```
INITIATED -> PROCESSING -> SUCCESS
          |
          +--> FAILED
          +--> UNKNOWN -> RECONCILIATION
```

123. Design a Notification Service.

Answer / Explanation: Accept a notification command/event, persist delivery state, route by channel, retry transient provider errors, use a DLQ, and make delivery idempotent where provider semantics permit.

```
Kafka
-> Notification Service
    -> Email
    -> SMS
    -> Push
```

124. Design a URL shortener.

Answer / Explanation: Generate a unique short code, store code-to-URL mapping, expose redirect API, cache hot mappings, define expiration, and consider collision handling and high read volume.

```
POST /urls -> {shortCode}
GET /{shortCode} -> 302 Location: originalUrl
```

125. Design a rate limiter.

Answer / Explanation: Token bucket supports bursts with a refill rate. Store counters/tokens in a shared low-latency store for horizontally scaled instances and return HTTP 429 when the limit is exceeded.

```
tokens += refillRate * elapsed
if tokens >= 1:
    tokens--
    allow()
else:
    reject(429)
```

126. Design a file upload service.

Answer / Explanation: For large files, avoid routing all bytes through application memory. Use pre-signed object-storage URLs, metadata persistence, antivirus/content validation and asynchronous processing.

```
Client -> Upload API
        -> pre-signed URL
        -> Object Storage
        -> Event -> Scan/Process
```

127. Design an audit logging system.

Answer / Explanation: Capture immutable actor/action/resource/time/correlation information. Prefer asynchronous publication for high-volume logging, but ensure security-critical audit events have reliable persistence and controlled retention.

```
service -> audit event -> Kafka
                        -> Audit DB/Object Store
```

128. Design a banking transfer workflow.

Answer / Explanation: Keep monetary state changes strongly consistent within the owning ledger boundary. Use idempotency, transaction boundaries, concurrency control, immutable ledger entries, reconciliation and explicit transaction states.

```
Transfer
-> validate
-> debit ledger
-> credit ledger
-> commit
-> publish event
```

129. Design a high-throughput search API.

Answer / Explanation: Separate transactional storage from search indexing when needed. Publish changes asynchronously, index documents, cache hot queries, paginate with search-after/cursors and monitor indexing lag.

```
DB -> Outbox -> Kafka -> Search Index
Client -> Search API -> Index
```

130. Design an inventory reservation service.

Answer / Explanation: Reservation must prevent overselling under concurrency. Use atomic database updates or carefully designed distributed coordination, expiration of reservations and compensating release when orders fail.

```
UPDATE inventory
SET available = available - 1
WHERE sku = :sku AND available > 0;
```

12. Advanced Senior-Level Scenarios

These are the questions to practice aloud. Interviewers are testing judgment, not memorized annotations.

131. A downstream service returns 500 for 30 minutes. What should your service do?

Answer / Explanation: Fail fast after bounded timeout, open the circuit, use a safe fallback if the business permits, queue asynchronous work if appropriate, and avoid retry storms. Surface an explicit degraded state rather than hiding the failure.

```
timeout=1s
retry=limited
circuit=open
fallback=DEGRADED
```

132. Your service receives the same request five times. How do you handle it?

Answer / Explanation: Determine whether the operation is idempotent. For non-idempotent business operations, use a client-provided or server-generated idempotency key with a durable unique constraint and stored result.

```
UNIQUE(idempotency_key)
status=SUCCESS
response_json={...}
```

133. Two services need the same customer data. Should they share a database?

Answer / Explanation: Usually no. One service should own the customer data and expose an API/event or replicated read model for consumers. Shared tables create coupling and make independent deployment difficult.

```
Customer Service -> source of truth
                  -> CustomerUpdated event
                  -> Consumer read models
```

134. How would you migrate Oracle to PostgreSQL with minimal downtime?

Answer / Explanation: Inventory SQL/data types and behavior, identify incompatible functions, build a migration pipeline, run dual validation or controlled replication, perform rehearsals, cut over with a rollback plan and verify business-critical queries.

```
Oracle
-> CDC/ETL
-> PostgreSQL
-> validation
-> controlled cutover
```

135. How do you handle backward-compatible API evolution?

Answer / Explanation: Add fields rather than changing existing meanings, tolerate unknown fields where appropriate, introduce new endpoints/version only for breaking changes, and publish a deprecation schedule.

```
v1 response remains valid
v2 adds:
"preferredName": "..."
```

136. What happens if Kafka is down when your transaction commits?

Answer / Explanation: Do not commit the business transaction and then hope a synchronous publish succeeds. Use a transactional outbox so the event is stored with the business transaction and published later with retries.

```
BEGIN
business_update()
outbox_insert()
COMMIT
```

```
publisher -> Kafka
```

137. How do you prevent a retry storm?

Answer / Explanation: Use short bounded retries, exponential backoff, jitter, circuit breakers, retry only transient errors, and respect a global request deadline. Never retry blindly at every layer.

```
attempt 1: 100ms
attempt 2: 250ms
attempt 3: 600ms
+ jitter
```

138. What if a request times out but the server actually completed the operation?

Answer / Explanation: The client may retry, creating a duplicate. This is a classic reason for idempotency keys and query/status endpoints for long-running operations.

```
POST /payments
Idempotency-Key: abc123
```

```
GET /payments/status?key=abc123
```

139. How do you handle a cache stampede?

Answer / Explanation: When a popular key expires, many requests can hit the database simultaneously. Use jittered TTLs, request coalescing/single-flight, background refresh or stale-while-revalidate strategies.

```
cache miss
-> one request loads DB
-> others wait/share result
```

140. Cache-aside vs write-through?

Answer / Explanation: Cache-aside loads data into cache on read miss and updates the DB first; it is simple but has consistency windows. Write-through updates cache as part of the write path, improving read freshness at the cost of more complexity.

```
read:
cache.get -> miss -> DB -> cache.put
```

141. How do you handle a thundering herd during service startup?

Answer / Explanation: Use readiness gates, gradual rollout, connection-pool limits, jittered scheduled work, cache warm-up with controlled concurrency and avoid every replica simultaneously performing expensive initialization.

```
replicas: 10
startup work: jitter 0-30s
```

142. What is zero-downtime deployment?

Answer / Explanation: Maintain capacity while replacing versions, use readiness probes, backward-compatible API/schema changes, graceful shutdown and health-aware load balancing.

```
v1 pods -> v2 pods
traffic only to READY pods
```

143. How do you safely shut down a Spring Boot service?

Answer / Explanation: Stop accepting new traffic, allow in-flight requests to finish within a deadline, stop consumers gracefully, commit/handle offsets correctly, and release resources.

```
SIGTERM
-> readiness=false
-> drain
-> stop consumers
-> shutdown
```

144. How would you design a service for 10,000 requests/sec?

Answer / Explanation: Start with workload characteristics and latency SLOs. Horizontally scale stateless instances, cache where useful, tune DB indexes/pools, partition workloads, use async processing for non-critical work, and load-test before selecting capacity.

```
Load Balancer
-> N stateless instances
-> cache
-> DB/read replicas
-> Kafka for async work
```

145. How do you choose between database locking and distributed locking?

Answer / Explanation: Prefer local database concurrency control when the resource is owned by one database. Distributed locks add failure modes such as lease expiry and split-brain risks; use them only when the coordination boundary truly spans resources.

```
UPDATE account
SET balance = balance - :amount
WHERE id=:id AND balance >= :amount;
```

146. How do you handle an external payment provider returning UNKNOWN?

Answer / Explanation: Do not mark it failed merely because the HTTP call timed out. Store PROCESSING/UNKNOWN, use provider reconciliation/status APIs, and make retries idempotent.

```
PROCESSING
-> UNKNOWN
-> provider status check
-> SUCCESS / FAILED
```

147. What should an error response contain?

Answer / Explanation: Use a stable machine-readable code, human-readable message, correlation ID and validation details where appropriate. Avoid stack traces, SQL and sensitive internal data.

```
{
  "code": "PAYMENT_TIMEOUT",
  "message": "Payment is being processed",
  "correlationId": "abc-123"
}
```

148. How do you manage configuration across environments?

Answer / Explanation: Keep application behavior in versioned configuration templates and inject environment-specific values through deployment configuration/secrets. Avoid branching source code by environment.

```
APP_ENV=prod
PAYMENT_TIMEOUT_MS=1000
```

149. How do you approach a production incident?

Answer / Explanation: First protect customers and stabilize the system. Establish incident ownership, inspect metrics/logs/traces, mitigate safely, communicate impact, preserve evidence, then perform a blameless root-cause analysis and preventive actions.

```
Detect -> Triage -> Mitigate -> Recover
      -> RCA -> Prevent recurrence
```

150. What makes someone a senior Java backend engineer?

Answer / Explanation: The ability to make sound trade-offs, design reliable APIs and boundaries, reason about concurrency/data consistency, debug production systems, communicate risks, automate delivery, and improve maintainability—not simply knowing more annotations.

Requirement

- > trade-offs
- > design
- > implementation
- > tests
- > observability
- > operations

13. Mock Interview — 10 Senior-Level Rounds

Practice answering each question aloud in 2–4 minutes. A strong answer should state assumptions, make a decision, explain trade-offs, cover failure modes and mention observability/testing.

Mock 1 — Architecture

Interviewer: Design an Order + Payment + Inventory platform.

Ideal answer approach: Ideal structure: clarify requirements and scale; define service boundaries and ownership; choose REST for immediate validation and Kafka for asynchronous events; use Saga/outbox for cross-service workflow; make payment idempotent; add circuit breakers/timeouts; secure APIs with OAuth2/JWT; add tracing/metrics/logging; deploy stateless services with Kubernetes and readiness probes.

Mock 2 — Production

Interviewer: Payment API latency increased from 300 ms to 6 seconds after a deployment.

Ideal answer approach: Start by checking the deployment diff and SLO dashboards. Use traces to determine whether time is spent in application code, database or provider calls. Check thread/connection pools, GC and CPU. Roll back if customer impact is high while investigation continues. After stabilization, reproduce/load-test and fix the measured bottleneck.

Mock 3 — Java Coding

Interviewer: Find the highest salary by department using streams.

Ideal answer approach: Use `groupBy` with `maxBy`, or `groupBy` plus mapping if only the salary is needed. Explain complexity and note that database-side aggregation may be preferable for millions of records.

Mock 4 — Microservices

Interviewer: How do you avoid duplicate payment after a client retry?

Ideal answer approach: Use an idempotency key with a unique database constraint. Store request state and final response. On duplicate requests, return the stored response. Make downstream provider calls idempotent where supported and reconcile UNKNOWN states.

Mock 5 — Kafka

Interviewer: A consumer is falling behind. What do you do?

Ideal answer approach: Measure lag and processing latency, identify slow downstream dependencies, inspect partition count and consumer parallelism, verify rebalances/errors, batch where safe, optimize processing, and scale consumers only when partitions and downstream capacity permit.

Mock 6 — Spring

Interviewer: Why did `@Transactional` not roll back?

Ideal answer approach: Check whether the method is invoked through the Spring proxy, whether self-invocation bypassed the proxy, whether the exception type triggers rollback, whether the transaction manager is correct, and whether the DB participates in the transaction.

Mock 7 — Database

Interviewer: A JPA endpoint generates 1 + 500 queries.

Ideal answer approach: Identify N+1 behavior. Inspect SQL logs/traces. Replace accidental lazy iteration with a deliberate fetch plan: fetch join, EntityGraph, DTO projection or batch fetching. Verify with query count tests and performance measurements.

Mock 8 — Security

Interviewer: How should microservices validate JWTs?

Ideal answer approach: Validate issuer, signature, expiration and audience at the appropriate trust boundary; map claims to authorities; use short-lived credentials; protect signing keys; and avoid trusting arbitrary user-supplied identity headers.

Mock 9 — System Design

Interviewer: Design a notification service handling millions of events/day.

Ideal answer approach: Use Kafka as a buffer, a notification service with channel-specific workers, provider adapters, retry with backoff, DLQ, idempotency/deduplication, delivery status storage, rate limits, observability and controlled replay.

Mock 10 — Leadership

Interviewer: A team wants to introduce microservices because the monolith is 'old'. What do you say?

Ideal answer approach: Ask what measurable problem microservices solve. If independent scaling/deployment or domain/team boundaries justify decomposition, start with a bounded capability and strangler approach. Otherwise, improve the monolith first. Explain the operational cost of distributed systems.

Rapid Revision Checklist

- Core Java: collections, equals/hashCode, immutability, concurrency, JVM and GC.
- Java 8+: lambda, functional interfaces, Stream API, collectors and CompletableFuture.
- Spring Boot: DI, lifecycle, configuration, profiles, validation, transactions and Actuator.
- REST: HTTP semantics, idempotency, pagination, versioning, errors and correlation IDs.
- Microservices: boundaries, service discovery, gateway, resilience, Saga and outbox.
- Kafka: partitions, keys, consumer groups, offsets, ordering, retries, DLQ and lag.
- Security: OAuth2/OIDC, JWT, authorization, service identity, secrets and rate limiting.
- JPA/SQL: N+1, transactions, locking, indexes, query plans and connection pools.
- Production: metrics, logs, tracing, thread dumps, heap dumps, latency and incident response.
- DevOps: Docker, CI/CD, Kubernetes, readiness/liveness and zero-downtime deployment.
- System design: requirements, scale, APIs, data, consistency, failure handling and trade-offs.